gemstone-rs Comparison

gemstone-rs Comparison Notes

This note is the JavaScript package's local summary of `gemstone-rs vs gemstone-py`. The source-of-truth Rust guide lives in the adjacent `gemstone-rs` repository as `docs/gemstone-py-vs-gemstone-rs.md`.

Run the packaged summary from this package with:

```
gemstone-js-compare gemstone-rs
gemstone-js-compare --target gemstone-rs --view scorecard
gemstone-js-compare gemstone-rs --scorecard
gemstone-js-compare gemstone-rs --batches
gemstone-js-compare --target gemstone-rs --scope beta --view totals
gemstone-js-compare gemstone-rs --scorecard --markdown
gemstone-js-compare gemstone-rs --totals --json
gemstone-js-compare --target gemstone-rs --view totals --json --assert-total-batches 6 --assert-hours-max
gemstone-js-compare --target gemstone-rs --scope beta --view totals --json --assert-total-batches 4 --asse
gemstone-js-compare --target gemstone-rs --view totals --max-total-batches 6 --max-hours-max 79 --quiet
gemstone-js-compare --target gemstone-rs --scope beta --view totals --max-total-batches 4 --max-hours-max
gemstone-js-compare gemstone-rs --scorecard --json --output gemstone-rs-comparison.json
gemstone-js-compare gemstone-rs --scorecard --format markdown --output gemstone-rs-comparison.md
gemstone-js-compare all --batches
```

Every `--json` report includes `$schema: "./schemas/comparison-report.schema.json"` and `schema_version: 1`, matching the packaged schema export for CI and editor tooling. `--output <path>` writes the selected JSON, Markdown, or human-readable report directly for release artifacts.

Short Answer

`gemstone-py` is still the more complete product for Python teams: it has the broader Python API, async examples, FastAPI/Litestar/Django coverage, the more mature database explorer, and the deeper PyPI/TestPyPI/native wheel release lane.

`gemstone-rs` is the better direction for Rust-native services, CLIs, workers, typed generated wrappers, explicit OOP/value handling, and the eventual shared native GCI core. It already has the safer long-term ownership boundary for GCI: keep unsafe calls isolated in Rust, then let higher-level language packages wrap that core.

Practical Choice

Use `gemstone-py` when:

- the application is Python-first
- you want the broadest examples and web framework coverage today
- you need the mature Python explorer and release lane now

Use `gemstone-rs` when:

- the application is a Rust service, CLI, worker, or local tool
- compile-time checked wrappers and typed mapping matter
- you want to move toward a shared Rust native core under `gemstone-py-native`

Remaining Work

For the Rust/Python track, the local Rust plan currently says **6 batches**, roughly **44-79 hours**:

1. Explorer and VS Code webview polish
2. Object mapping maturity
3. Codegen live discovery and generated tests
4. Async facade and web middleware
5. Shared core with gemstone-py-native
6. Release and live CI hardening

The combined JavaScript plus Rust ecosystem plan is **12 batches**, roughly **86-151 hours**. That number includes the broader gemstone-js product-polish track, not just the narrower JS beta-hardening estimate in `docs/gemstone-py-parity.md`.

For the narrower beta-hardening scope, run `gemstone-js-compare --target gemstone-rs --scope beta --view totals`. The local conservative Rust beta plan is **4 batches**, roughly **26-45 hours**.

Batch Interpretation

The most important `gemstone-rs` batch is still explorer/workbench polish. The Rust API has credible core coverage, but `gemstone-py` wins the user-facing experience because its explorer and VS Code workflow are more complete.

The highest-leverage architecture batch is shared core integration: `gemstone-py-native` should eventually become a thin PyO3 adapter over `gemstone-gci/gemstone-rs`, while `gemstone-py` keeps the idiomatic Python surface and `gemstone-rs` keeps the idiomatic Rust surface.